

[Sign Out](#)[Account](#)[Markov Decision Processes and Value Functions](#)[21 min read](#)

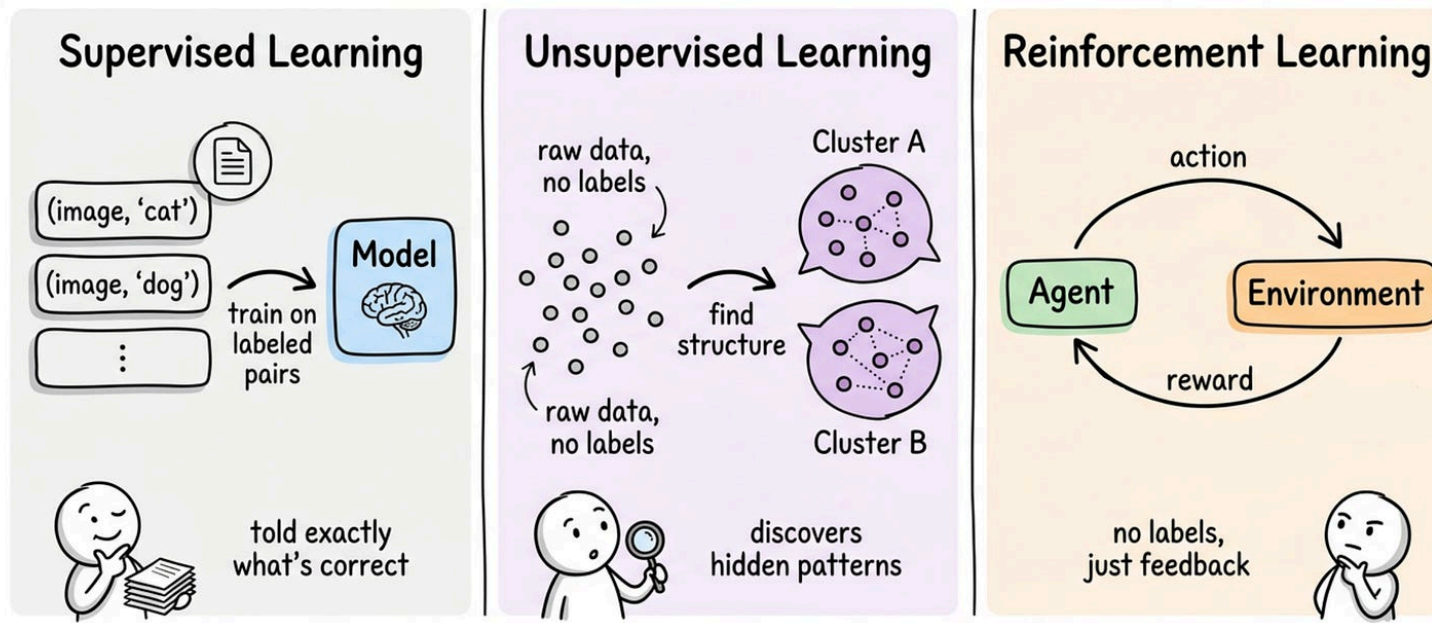
Markov Decision Processes and Value Functions

RL Part 2: Markov decision processes, returns, policies, and value functions.

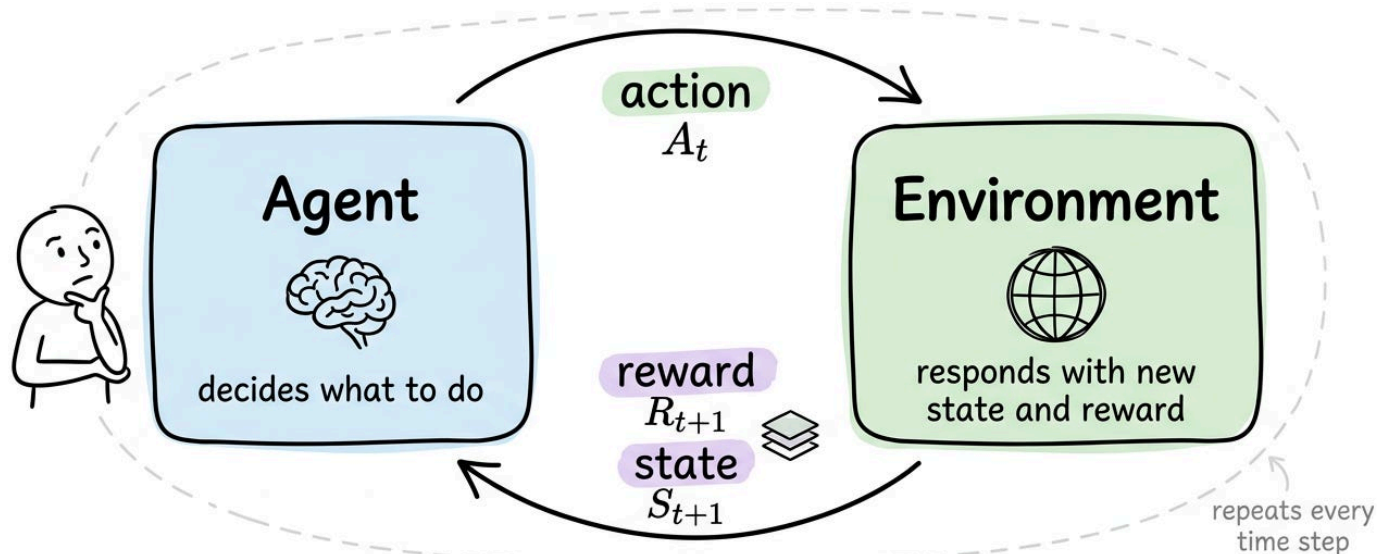
Recap

In the previous chapter, we introduced reinforcement learning as a third kind of machine learning, distinct from supervised and unsupervised learning.

Three ML Paradigms Compared

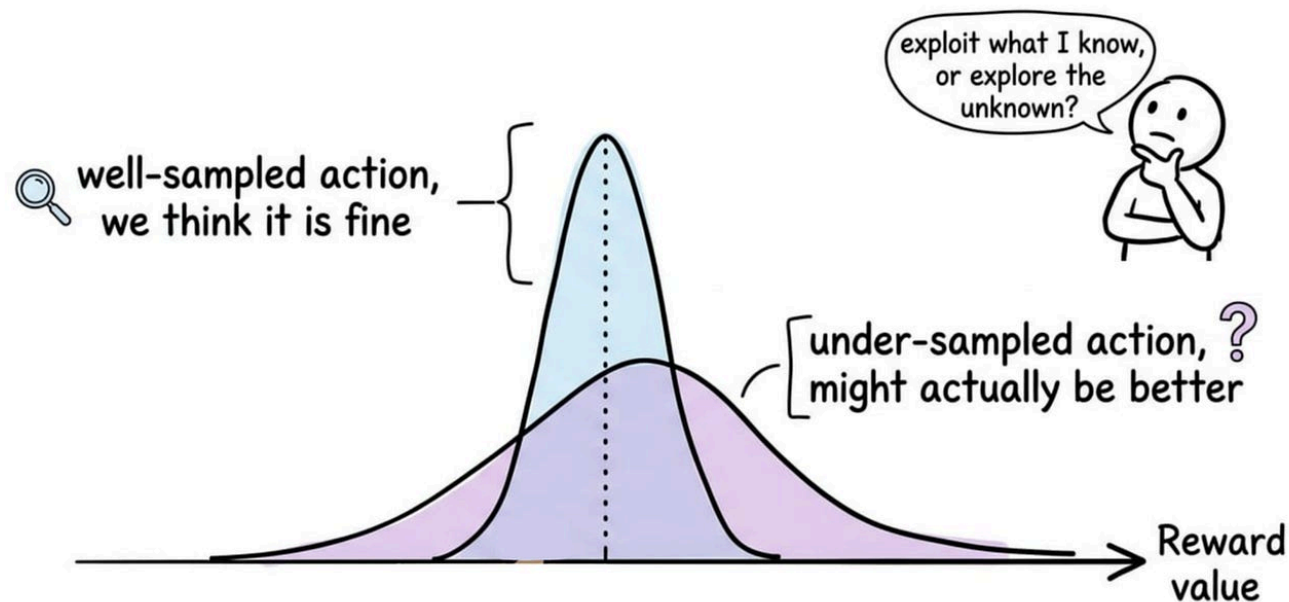


We saw the core agent-environment loop, where the agent picks actions, the environment returns rewards and a new state, and the cycle continues.

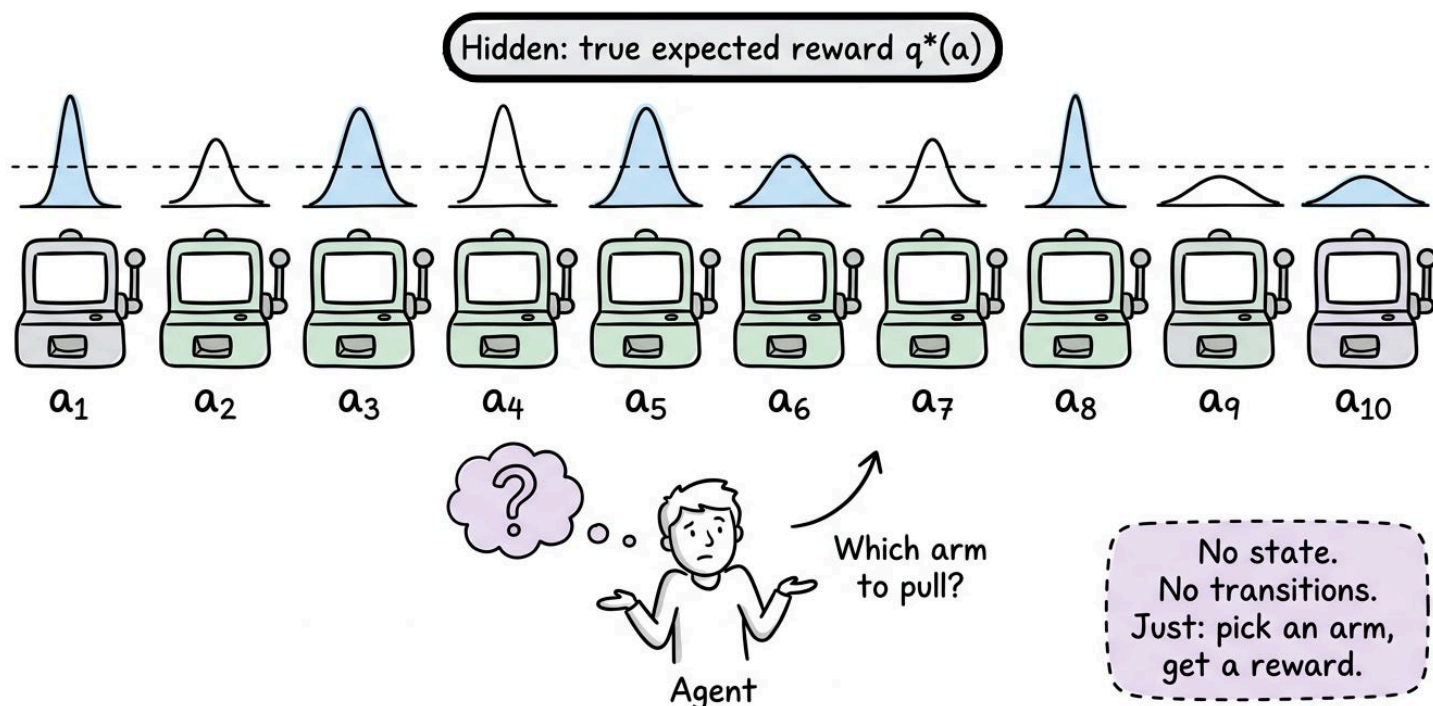


Every RL problem reduces to the same loop: the agent acts, the environment responds, and the cycle repeats.

We discussed four properties that make RL distinctive: feedback is evaluative rather than instructive, the data distribution depends on the agent's own behavior, consequences are delayed and create the credit assignment problem, and exploration cannot be avoided.

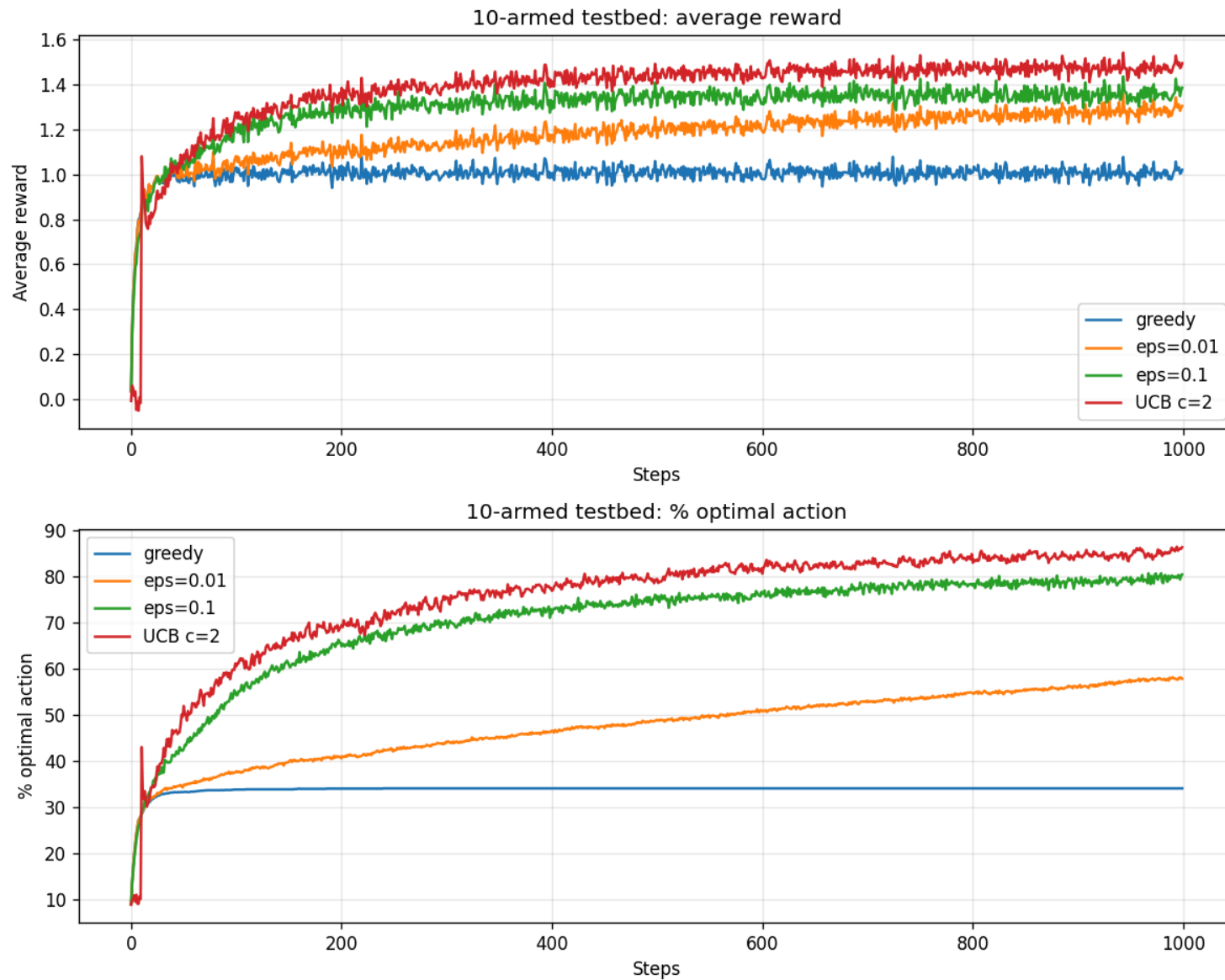


We then studied the simplest possible RL problem: the multi-armed bandit. With no states to track, only k actions and unknown reward distributions, the bandit isolates the exploration-exploitation tradeoff.



The multi-armed bandit strips RL down to its hardest idea: you must learn which arms pay more by pulling them, without ever seeing the true distributions.

We worked through four strategies, namely greedy, ϵ -greedy, optimistic initial values, and upper confidence bound (UCB), and ran them on the 10-armed testbed to see how each one trades off exploration against exploitation in practice.



Overall, the previous chapter familiarized us with some of the most basic ideas of reinforcement learning.

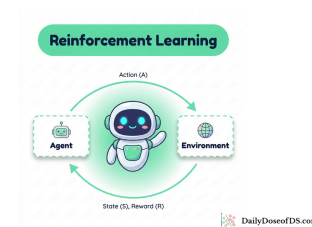
If you haven't yet gone through Part 1, we recommend reviewing it first, as it establishes the conceptual foundation essential for understanding the material we're about to cover.

Read it here:

Foundations of Reinforcement Learning

RL Part 1: Agents, environments, rewards, and why RL is different from supervised learning.

 Daily Dose of Data Science • Akshay Pachaar



In this chapter, we will understand and explore the Markov property, Markov decision processes (MDPs), value functions and much more.

As always, every notion will be explained through clear examples and walkthroughs to develop a solid understanding.

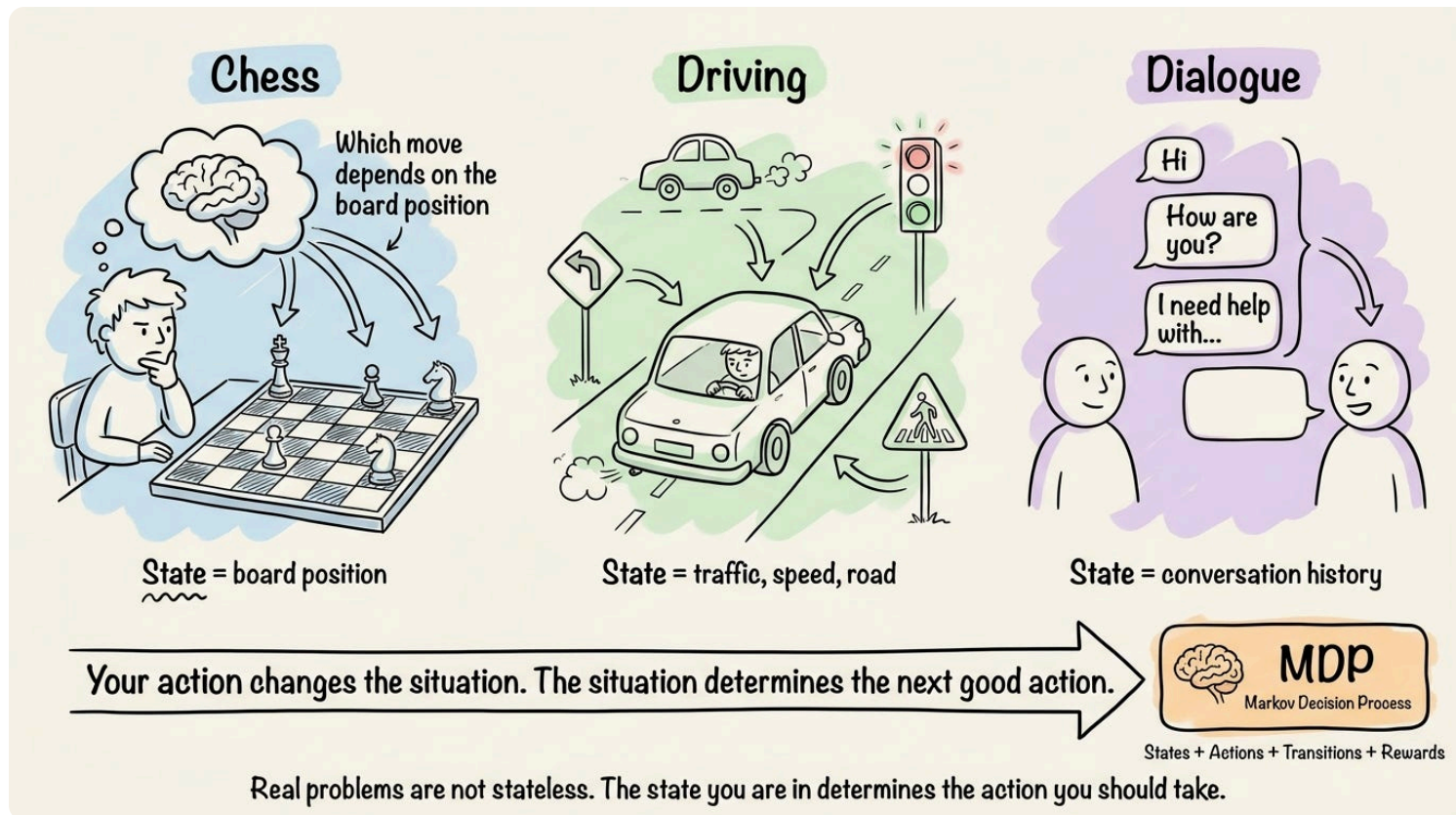
Let's begin!

Introduction

In Chapter 1, the bandit gave us a clean way to think about exploration and exploitation. But it hid a detail that changes a lot of things. Real problems are not stateless.

In chess, the move you should make depends on the current board position. In driving, the next steering action depends on the surrounding traffic. In a dialogue, what to say next depends on the conversation so far. The action you take changes the situation you face afterward, and the situation determines which actions are good.

To talk about states, transitions, and how rewards depend on both, we need a formal vocabulary.



That vocabulary is the Markov decision process (MDP), and it underpins essentially every algorithm we will study.

We will explore the framework of MDPs. We will start with the Markov property, which is what makes the whole edifice tractable. We will then formalize the MDP itself and discuss episodic versus continuing tasks. From there, we will define the return, discounting, and the reward hypothesis.

We will introduce policies as the object the agent actually learns, and define two value functions, v_π and q_π , that measure how good states and actions are under a given policy.

Finally, we will build a small gridworld from scratch and use Monte Carlo rollouts to estimate v_π for a fixed policy.

The Markov property

Before defining what an MDP is, we need the assumption it rests on: the Markov property.

Informally, a state is Markov if the future depends on the past only through the present. This means that once you know the current state, the entire history that led to it is irrelevant for predicting what happens next.

Formally, the state S_t is Markov if the distribution of the next state conditioned on the entire history reduces to a distribution conditioned only on the current state and action:

$$\Pr(S_{t+1} = s' \mid S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = \Pr(S_{t+1} = s' \mid S_t, A_t)$$

Distribution of the next state
given the entire history

Distribution of the
next state based
on the current
state/action



We are using Pr for generic probability statements, since we'll be using P for transition function.

Here, S_{t+1} is the resulting next state, and s' is a particular value the next state can take. The left side is the probability of landing in s' given the entire history of states and actions. The right side is the same probability, but conditioned only on the current state and action.

The Markov property says these two are equal.

This has enormous benefits.

Without it, predicting the next state would, in principle, require remembering everything the agent has ever seen. With it, we only need the current state. The

state is a complete summary of the past, sufficient to predict the future.

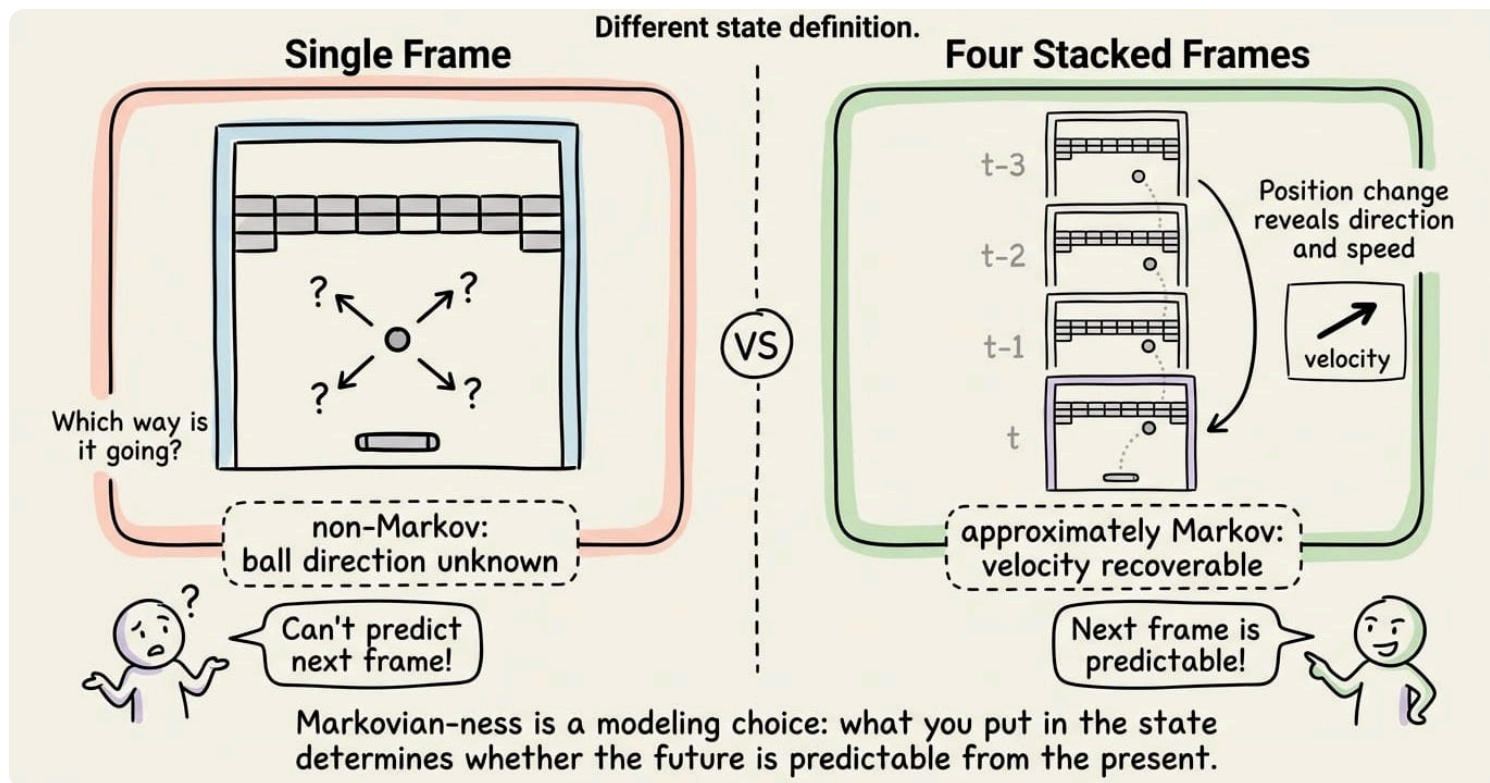
Here's a concrete example. Consider the Atari game Breakout, with the bat at the bottom and a ball bouncing around.



If we treat a single screenshot as the state, is the state Markov? The answer is no.

From one frame, you cannot tell which direction the ball is moving. There are at least four directions consistent with any given frame, and they lead to very different next states.

But what if the state is the past four frames stacked together? Now you can compute the ball's position and velocity, and the next frame becomes predictable. The state, redefined this way, is approximately Markov.



This points to something important. "Markovian-ness" is often a modeling choice, not a physical fact. The same physical environment can be Markov or not, depending on what you decide to put in the state.



The art of formulating an RL problem is, in large part, designing a state representation that makes the property hold, or hold well enough.

When the property genuinely cannot hold, the agent only sees observations that are partial summaries of a true underlying state it cannot directly access.

This setting has its own name, which is called the partially observable Markov decision process (POMDP), introduced by Karl Johan Åström in 1965 for the discrete state space.

In a POMDP, the true state can be called hidden, not in the sense of being deliberately concealed, but in the sense that the agent only sees it indirectly through observations. POMDPs add an observation set and an observation function, and require the agent to track a probability distribution over what the hidden state might be, called a belief.



Note that we will not be studying POMDPs in depth, but it is worth knowing they exist, since many real-world problems are partially observable in nature.



A useful rule of thumb: if you find yourself wanting to add "history" to the agent's input, that is a sign your current state is not Markov. The fix is to enrich the state representation, not to give the agent memory tricks.

At this point, we must also understand that the Markov property does not say states are deterministic. The next state is allowed to be random. The property only says the randomness depends solely on the current state and action.



"Memoryless" is sometimes used as a synonym for Markov. The two mean the same thing here: the past history can be discarded once the current state is known.

With the Markov property in place, we can now formalize the agent-environment interaction as a tuple of components, each playing a specific role.

The MDP framework

A Markov decision process is a 5-tuple:

$$(\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

Each piece has a precise meaning. Let's go through them one at a time.

- $\mathcal{S}$ is the set of states, called the state space. It can be finite (every cell of a 4×4 grid), countably infinite (every legal chess position), or continuous (every joint angle of a robot arm).



We will mostly stay in the finite-state setting in the early chapters, since it makes algorithms easier to reason about and analyze.

- $\mathcal{A}$ is the set of actions, called the action space. In a grid (or gridworld) it might be {up, down, left, right}. In chess it is the set of all legal moves from the current position. Like the state space, the action space can be finite, countable, or continuous.
- P is the transition function. Given a current state and action, it specifies the probability distribution over next states:

$$P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a)$$



The Markov property is baked into this definition: the transition depends only on (s, a) , not on anything earlier.



The transition function P is sometimes called the dynamics.

- R is the reward function. There are two common formulations:
 - $R(s, a)$ as the expected reward for taking action a in state s
 - $R(s, a, s')$ when the reward also depends on the resulting next state

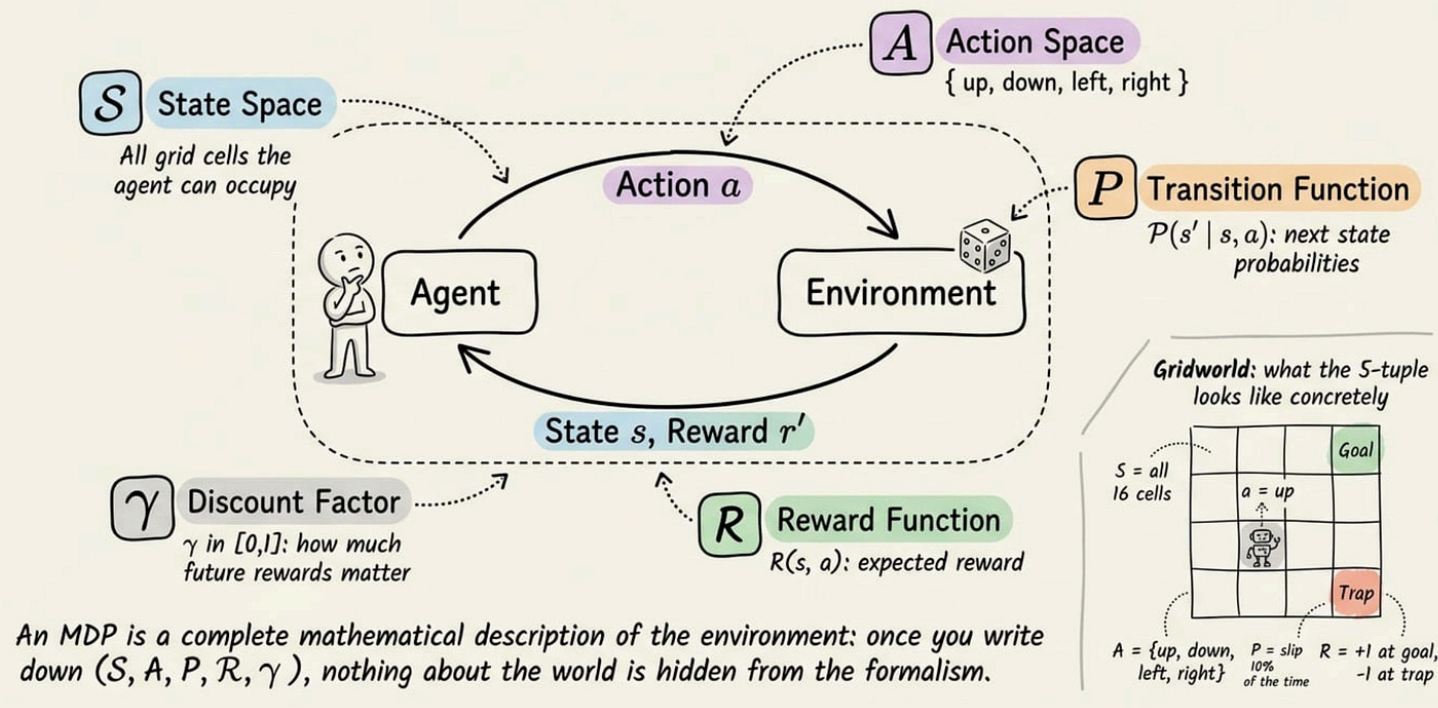


Both are widely used; the differences usually do not matter mathematically.

- γ is the discount factor, a real number in $[0, 1]$. It controls how much the agent values future rewards relative to immediate ones. We will dedicate a full section to it shortly, since it deserves more than a single line.

A useful way to think about this is that an MDP is a complete mathematical specification of the environment. Once you have written down (S, A, P, R, γ) , the environment has no remaining unspecified parameters. Anything left for the agent to figure out is internal to the agent: which actions to take in which states.

MDP 5-Tuple Components Around the Agent-Environment Loop



It is worth pausing on what this framework includes:

- Stochasticity is built in, since P is a probability distribution rather than a deterministic function.
- Sequential dependency is built in, since the next state depends on the current one.

- Goal-directedness is built in, since the agent is trying to maximize cumulative reward.



"An MDP is solved when we know its optimal policy." This phrase will recur throughout. For now, just keep it in mind. We will define what "optimal" means precisely once we have the value functions in hand.

It is also worth noticing that the bandits we studied in chapter 1 are MDPs. They are the special case where $|S| = 1$. With a single state, the transition function is trivial (you always end up in the same state), and the reward depends only on the action. The exploration-exploitation tradeoff we wrestled with there is the same one we will wrestle with in larger MDPs, just without the complication of states.



Technically, in the MDP formalism, the bandit is a single-state MDP. But since that one state never changes and carries no information, we informally/casually describe it as "stateless".

Now that we understand the MDP framework, there is one quick distinction worth pausing on before we move to how interactions unfold over time. The MDP contains P and R , but it does not say whether the agent has access to them.

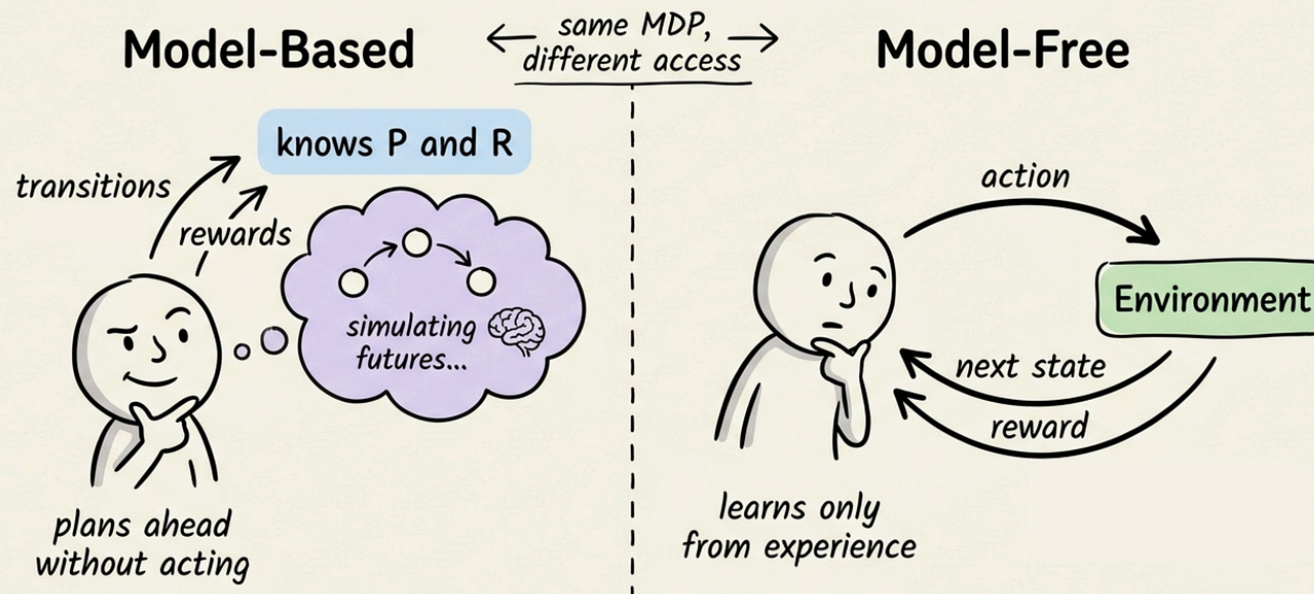
Model-based and model-free learning

A "model" here means knowledge of the environment's dynamics, namely the transition function P and the reward function R . The MDP we just defined contains both. The question is whether the agent has access to them.

In the model-based setting, the agent has P and R in hand. It can simulate transitions in its head, plan ahead, and compute outcomes.

In the model-free setting, the agent does not have P and R . It only has access to samples drawn from interaction: take an action, observe what state you land in and what reward you get. Whatever the agent learns about the environment, it learns from this stream of experience.

Model-Based vs Model-Free RL Agent



Model-based agents plan in their heads; model-free agents learn only from what they experience.



A useful mental model: model-based RL is "learn or know the rules, then plan". Model-free RL is "ignore the rules, just learn from playing". This essentially means, model-free learns after trying, but model-based can even decide before trying.

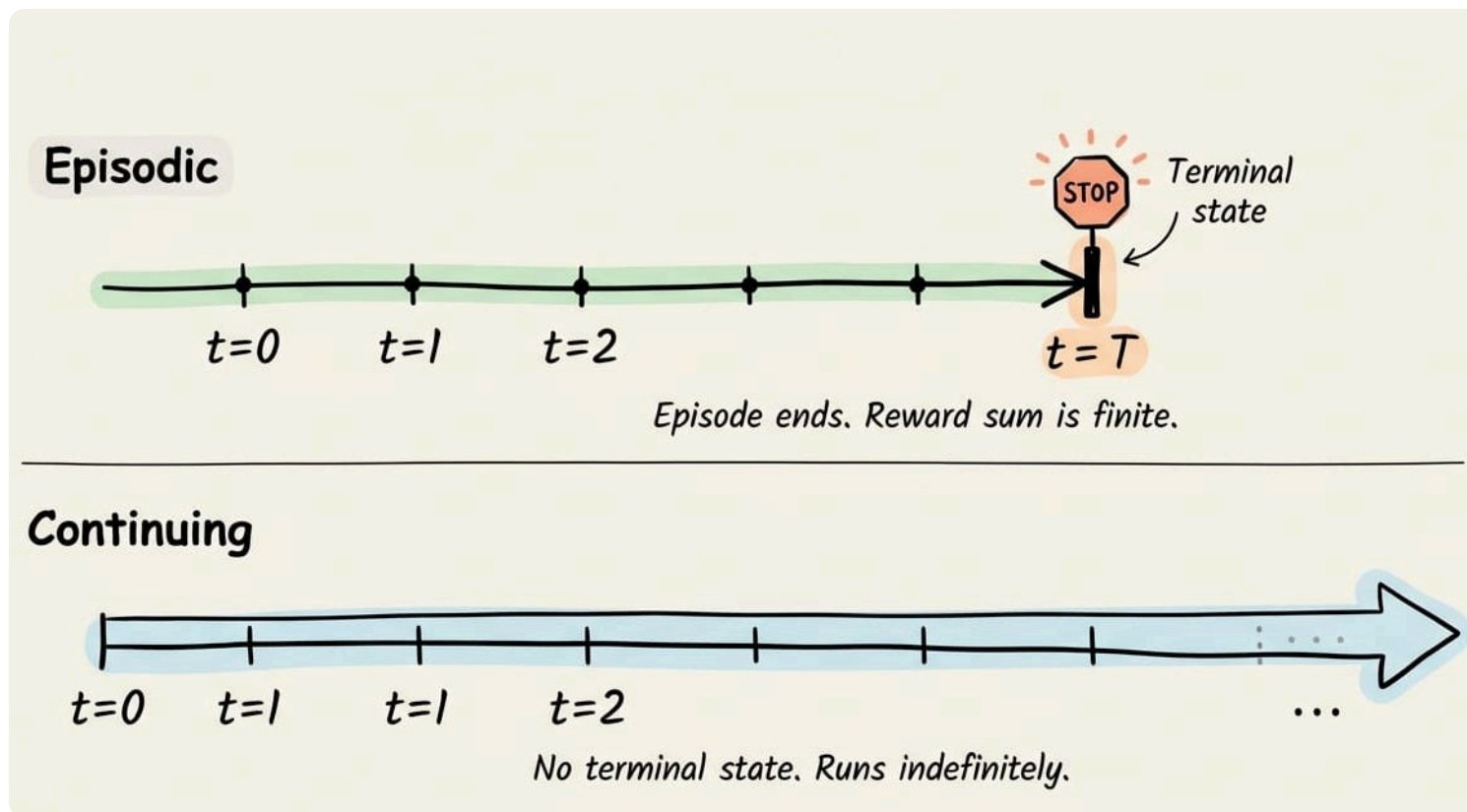
With the model-based and model-free distinction in mind, we can now turn from the static description of the environment to how interactions unfold over time. The first thing to pin down is whether those interactions terminate or run indefinitely.

Episodic and continuing tasks

The agent-environment loop unfolds over time. It can either run indefinitely or naturally come to an end.

In an episodic task, the interaction breaks into self-contained episodes. Each episode starts from some initial state, runs for a finite number of steps, and ends in a terminal state. For example, a chess game is episodic, and similarly a gridworld with a goal cell is episodic.

In a continuing task, the interaction has no natural endpoint. A robot balancing a pole forever or a recommender system serving an endless stream of users. These run, in principle, indefinitely. There is no terminal state, and the agent keeps acting and receiving rewards without bound.



The distinction matters because it changes how we define the agent's objective:

- In an episodic task, summing rewards from the start of an episode to its end is well-defined; the sum is finite because the episode is finite.
- In a continuing task, simply summing rewards forever is useless as an objective. We need a way to keep the sum bounded, which is what discounting will do for us (we'll learn more on it in the next section).



Some tasks can either be continuing or episodic depending on how you set up the state.

With the episodic versus continuing distinction in hand, we can now define the agent's actual objective: the return.

Returns and discounting

The agent picks actions over time and receives rewards $R_1, R_2, R_3, \dots$. To talk about its goal, we need a single number that summarizes the future. That number is the return, written G_t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Unpacking:

- G_t is the return starting at time t .
- The first term, R_{t+1} , is the reward received from the action taken at t .
Similarly the R_{t+2} , but weighted by γ and so on.

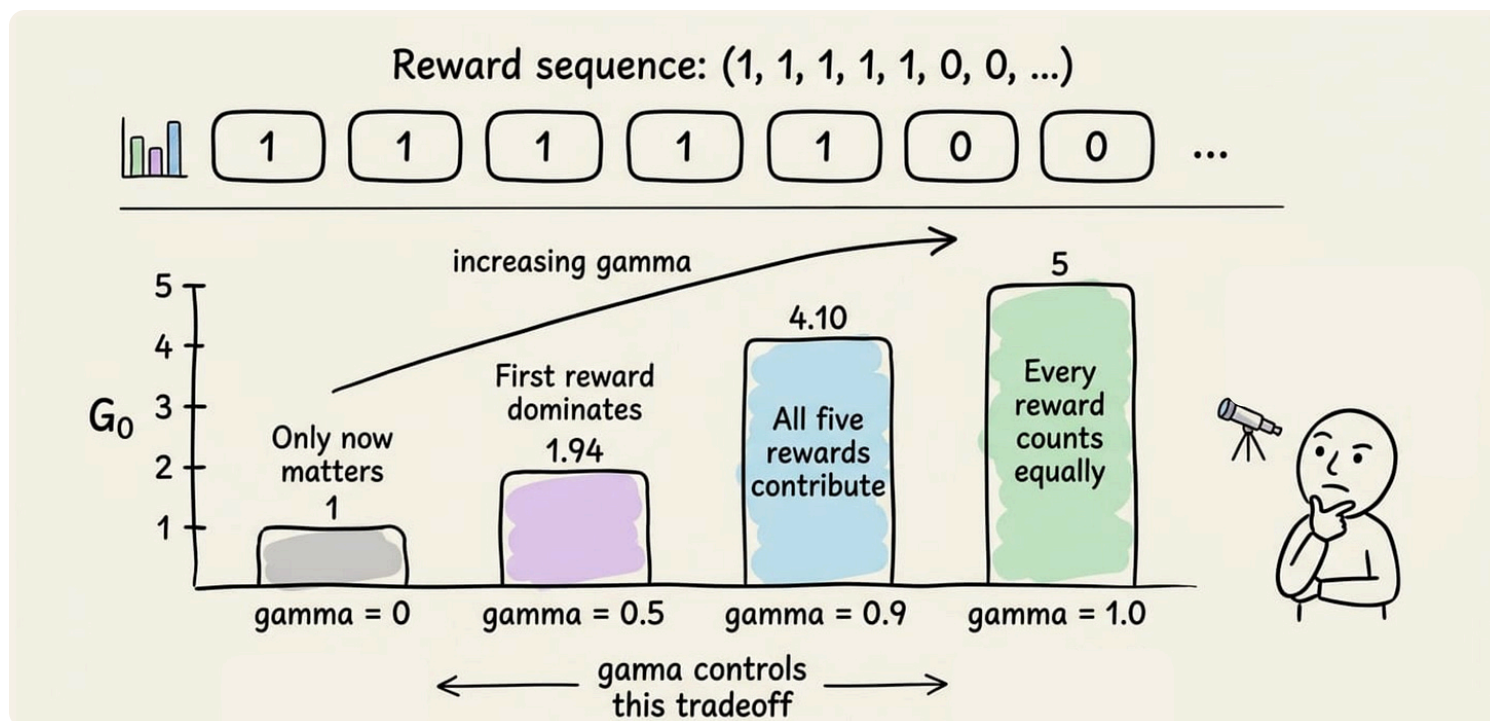
The discount factor $\gamma \in [0, 1]$ is what turns this from just a summing exercise into a flexible objective.

To see how it shapes behavior, consider a fixed reward sequence $(1, 1, 1, 1, 1, 0, 0, \dots)$, where the agent receives reward 1 for the next five steps and zero forever after.

Under different choices of γ , the return G_0 comes out very differently:

- $\gamma = 0$: $G_0 = 1$. Only the immediate reward counts; everything else is multiplied by zero.
- $\gamma = 0.5$: $G_0 = 1 + 0.5 + 0.25 + 0.125 + 0.0625 = 1.9375$. The first reward dominates; later rewards add diminishing contributions.
- $\gamma = 0.9$: $G_0 = 1 + 0.9 + 0.81 + 0.729 + 0.6561 \approx 4.10$. All five rewards contribute substantially.

- $\gamma = 1$ (undiscounted): $G_0 = 5$. Every future reward counts the same as an immediate one.

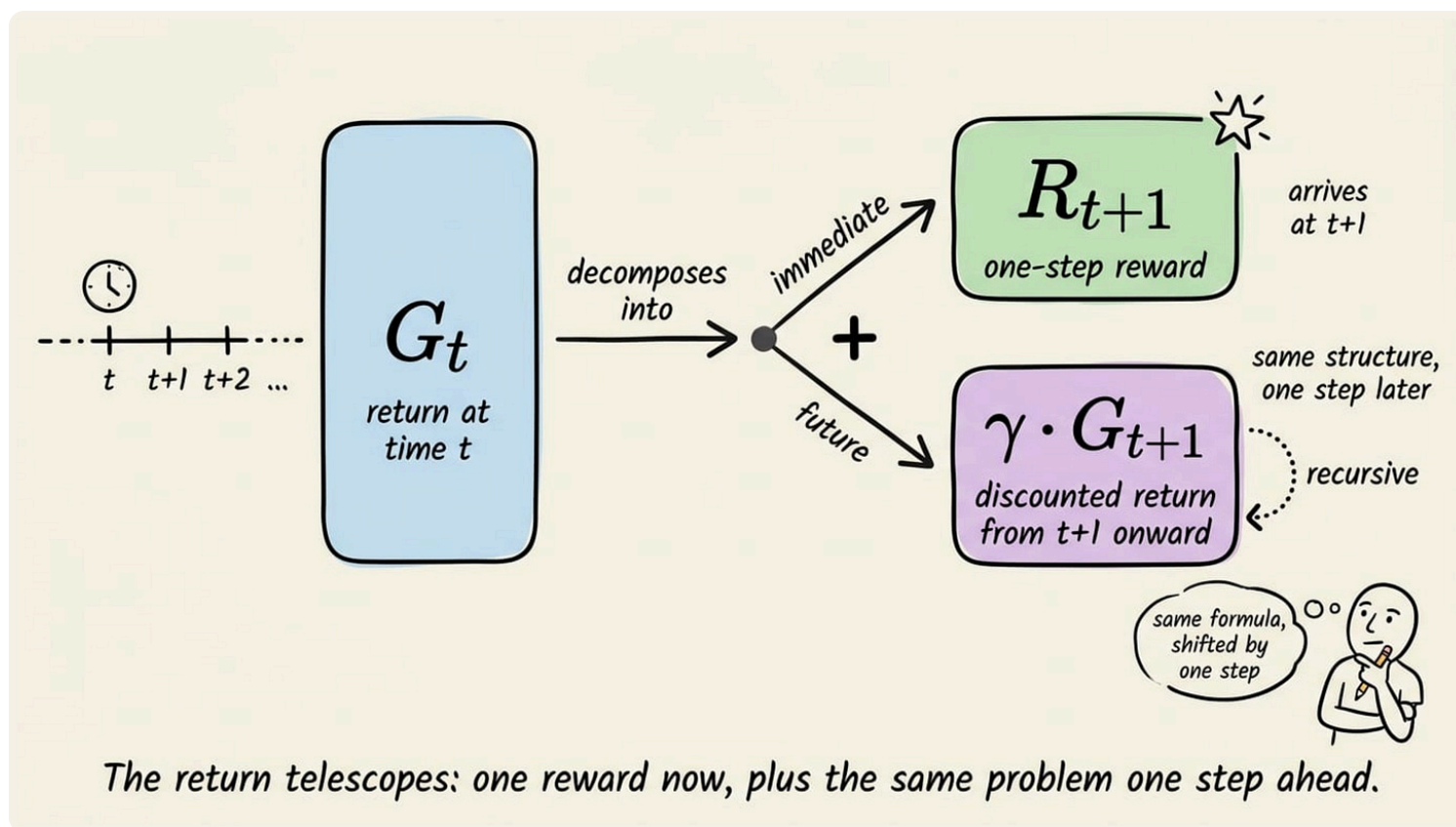


The intuition is that γ controls how far-sighted the agent is. A small γ produces a myopic agent that grabs whatever reward is immediately available. A large γ produces a far-sighted agent that values long-term gains.

The return also has a useful recursive structure. From the definition,

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) = R_{t+1} + \gamma G_{t+1}$$

This says the return at time t is the immediate reward plus the discounted return from the next time step. The agent's long-term goal decomposes cleanly into "the thing that happens" plus "everything after that".





For episodic tasks, you can use $\gamma = 1$ (no discounting) safely; the episode ends in finite time so the return is bounded. For continuing tasks, use $\gamma < 1$. Otherwise the return can be infinite and the optimization problem becomes ill-defined.

With returns defined, we now have a way to describe what the agent is trying to maximize over time. But this raises a deeper question: why should cumulative reward be the right objective in the first place?

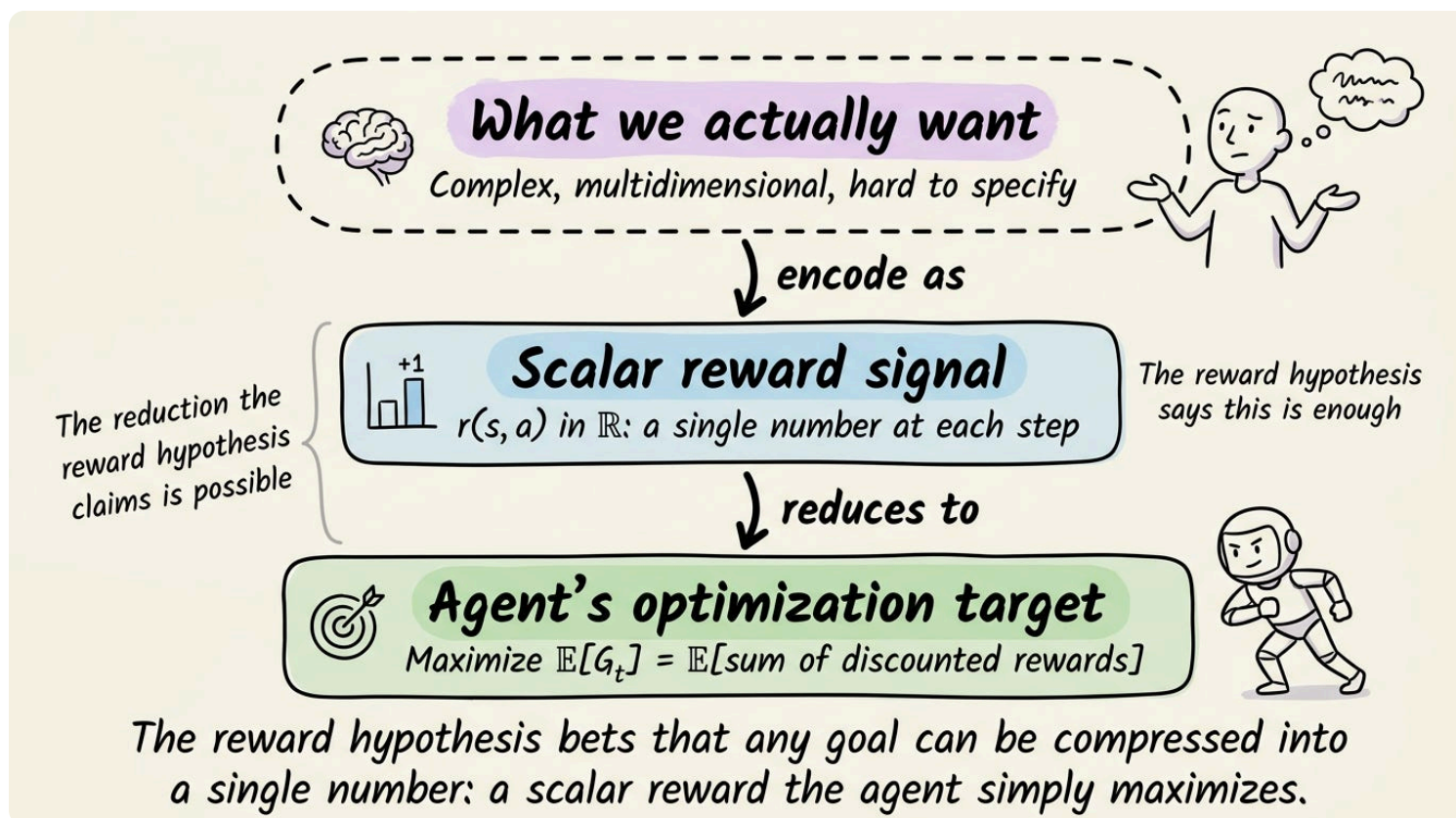
The reward hypothesis

The entire RL framework rests on a single bold claim, often called the reward hypothesis. Sutton and Barto formulate it in a way that basically says, any goal can be captured as the maximization of the expected cumulative sum of a scalar reward.

This is a strong commitment. It says that whatever we want the agent to do, we can encode that goal as a scalar reward signal. The agent's job then reduces to

maximizing the expected cumulative discounted sum of that signal.

The hypothesis is what justifies the entire structure we have built. It is why we can write down a single number G_t and call its expectation the agent's objective.



It is worth being honest about its limits, however. The hypothesis is a hypothesis, not a theorem. Recent studies show that scalar rewards can capture a fairly broad class of preferences, but only under certain rationality assumptions.

Real human preferences may not always satisfy them. Multi-objective settings, where the agent must trade-off, say, cost against safety against latency, push against the scalar formulation; workarounds exist, but they break some of the elegance of the standard framework.

Even when the hypothesis holds in principle, designing the reward function in practice is hard. A small misspecification can lead to behavior that maximizes the reward as written but violates the spirit of the task. This failure mode is called reward hacking, and it has become a central concern in modern RL training.

We will return to reward hacking and reward design in a later chapter. For now, the takeaway is that the reward hypothesis gives us a clean foundation, but the burden of correctly specifying the reward is non-trivial and is exactly where many real-world RL systems break.



Goodhart's law applies here: when a measure becomes a target, it ceases to be a good measure. Reward hacking is Goodhart's law applied to RL.

We have now defined the environment, the return, and the underlying assumption that justifies maximizing it. Now let's understand the missing piece: the agent's behavior. How does the agent decide what to do?

Policies

A policy is the agent's behavior rule. It maps states to actions, or more generally to distributions over actions.

There are two flavors:

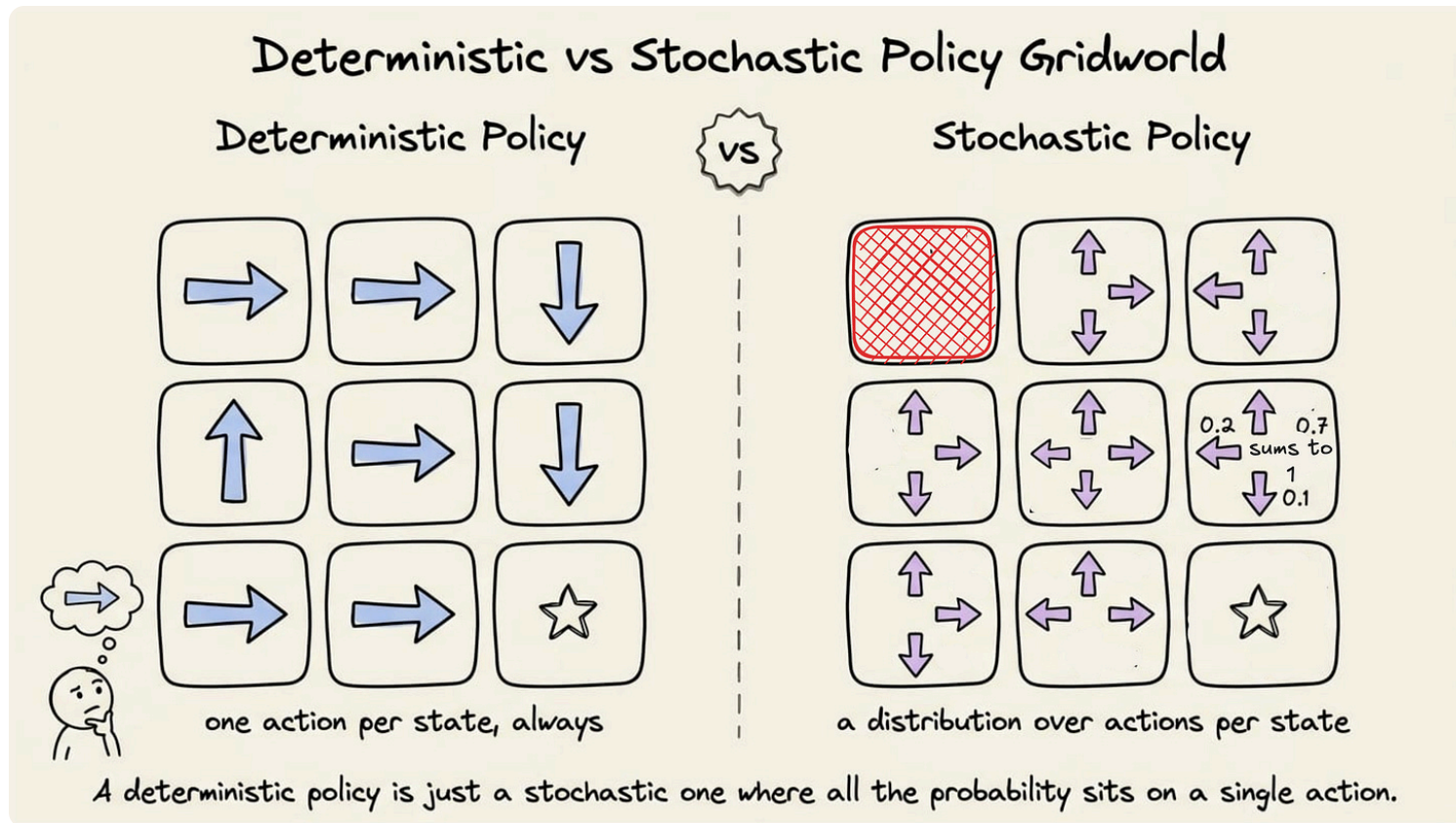
- A deterministic policy, written $\pi(s) = a$, is a lookup table (in state s , take action a). There is no randomness in the agent's decision; the same state always produces the same action.

- A stochastic policy, written $\pi(a \mid s)$, is a probability distribution over actions for each state:

$$\pi(a \mid s) = \text{Pr}(A_t = a \mid S_t = s)$$



For each state s , the values $\pi(a \mid s)$ across all actions sum to 1.



We now have the environment, the return, and the policy. The next step is to define what "good" means for a policy. That is the job of value functions.

State-value function

Given a policy, we want to ask: starting from state s and following π forever after, how much total reward do I expect to collect?

The answer is the state-value function, written $v_\pi(s)$:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

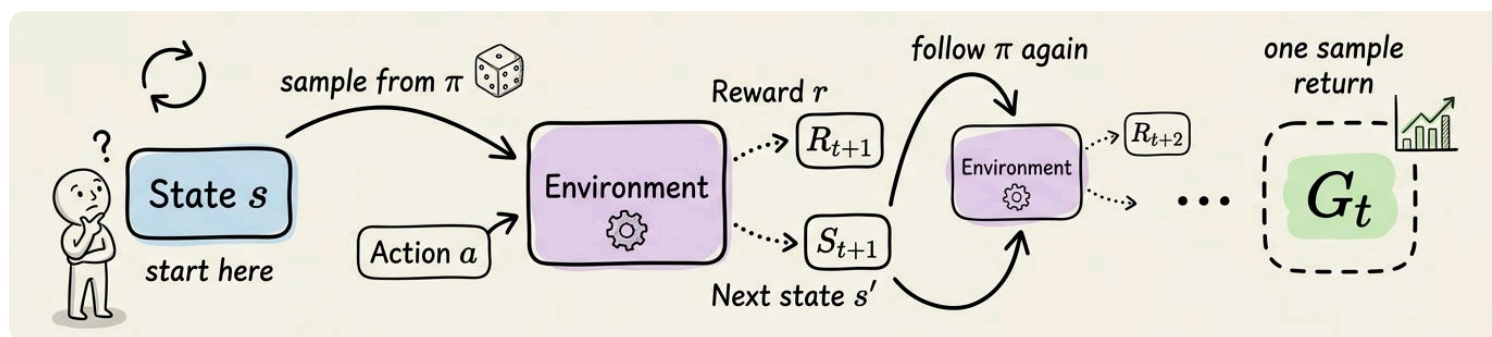
Unpacking:

- $v_\pi(s)$ is the expected return starting from state s and following π .
- The expectation $\mathbb{E}_\pi$ is taken over two sources of randomness:
 - the actions sampled from the policy $\pi(\cdot \mid \cdot)$
 - the next states sampled from the environment's transition function P
- The condition $S_t = s$ pins down the starting state.



Important note: values are policy-dependent.

Spelling out the structure: starting in s , the agent samples an action from π , the environment returns a reward and a next state, the agent samples another action from π , and so on. Each rollout (one complete run of the policy from s to termination) produces a return G_t , which is itself random. $v_\pi(s)$ is the average of those returns over all the randomness, conditional on starting in s .



The crucial thing to internalize is that values depend on the policy. The same state can be valuable under one policy and worthless under another.

Note: Terminal states have value zero by convention, since no further rewards can be collected from them.

Now that we have v_π for evaluating states, the natural next question is: what about evaluating actions? That is the action-value function.

Action-value function

The action-value function $q_\pi(s, a)$ measures the expected return from taking action a in state s and then following π forever after:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Unpacking:

- $q_\pi(s, a)$ is the expected return starting from state s , with the first action forced to be a , and following the policy π for every action thereafter.
- The expectation is taken over the same two sources of randomness as before: the policy's action distributions for all subsequent steps, and the

environment's transition probabilities throughout.

The difference from v_π is subtle but important. $v_\pi(s)$ assumes the agent samples its first action from π as well. But $q_\pi(s, a)$ pins down the first action and then defers to π . Thus, the two are linked as:

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a)$$

In words: the value of state s under policy π is the policy-weighted average of the action-values $q_\pi(s, a)$ across all possible actions.

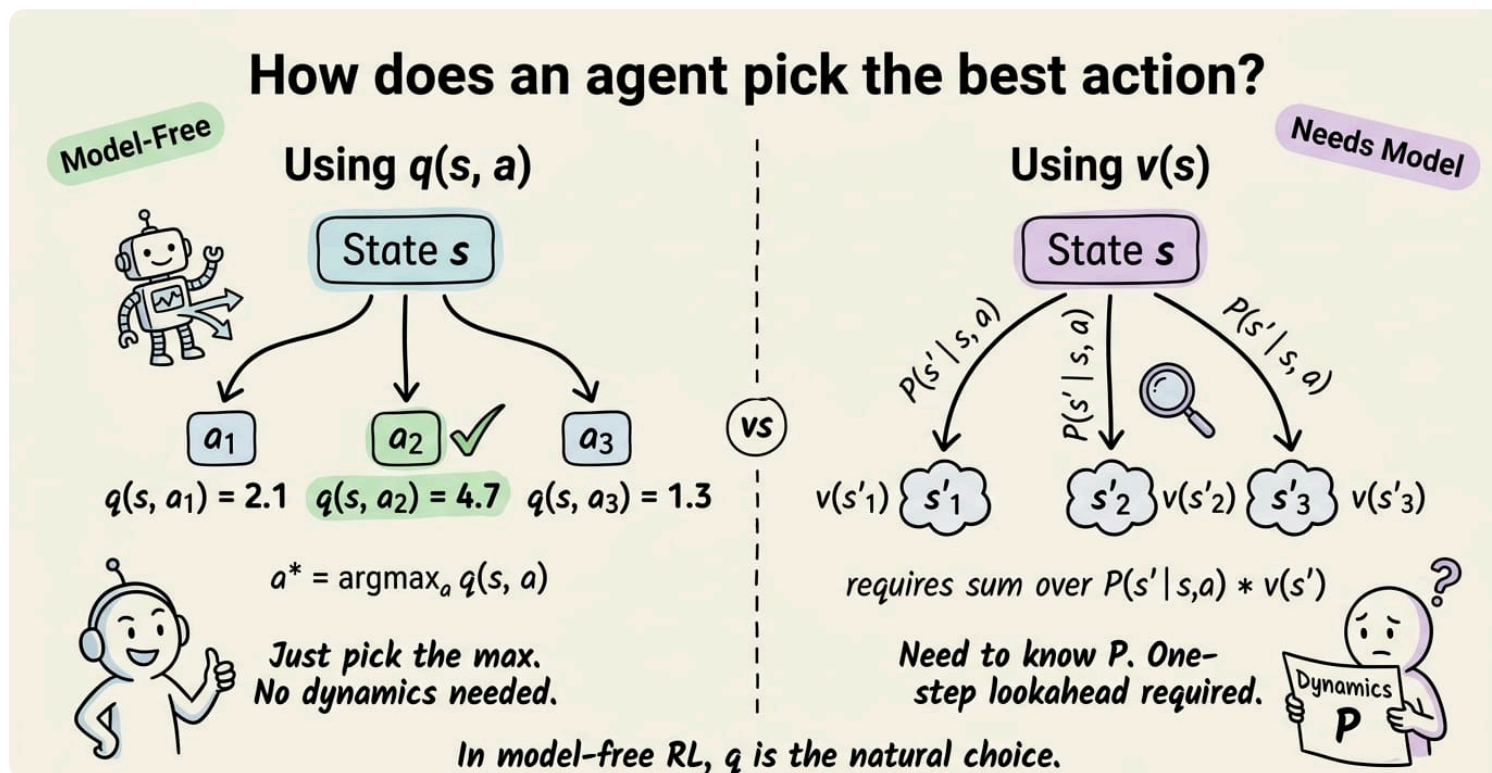
Now a question arises, why bother with q_π when v_π already exists? The answer is action selection. If you know $q_\pi(s, a)$ for every action in state s , picking the best action is just an arg-max:

$$a^* = \arg \max_a q_\pi(s, a)$$

The beauty is you do not need the environment dynamics to do this. With q_π , the values per action are right there, just pick the best one and you're done.

By contrast, v_π only gives values per state, not per action. To pick the best action from v_π , you'd need to look at every action's possible next states, weight each by its transition probability $P(s' \mid s, a)$, and sum up, which requires knowing P .

The transition function is exactly what's typically unavailable in real problems. This is why q_π is the natural object to estimate in model-free RL: it absorbs the dynamics into itself, so the agent never has to represent them explicitly.



$q_\pi(s, a)$ is also called the "Q-function" or "Q-value".

We have now defined everything: states, actions, dynamics, rewards, returns, policies, and the two value functions. The pieces are in place. Now let's bridge to the hands-on section.

Hands-on: gridworld and Monte Carlo policy evaluation

We have defined v_π as an expectation, but how do we actually compute it?

One natural answer: simulate many episodes under π , compute the return from each visited state, and average. The averaged return converges to v_π by the law of large numbers. This is called Monte Carlo (MC) policy evaluation.

Setup

The code and project setup we are using are attached below as a zip file. You can simply extract it and run `uv sync` command to get going.

```
.python-version  
gridworld.py  
main.py  
mc_eval.py  
pyproject.toml  
uv.lock  
value_heatmap.png
```

Download the zip file below:

gridworld-mc

gridworld-mc.zip • 71 KB



For details about the versions and dependencies, you can check the `.python-version` and `pyproject.toml` files.



It is recommended to follow the explanation ahead side-by-side with the above attached project for a more comprehensive understanding.

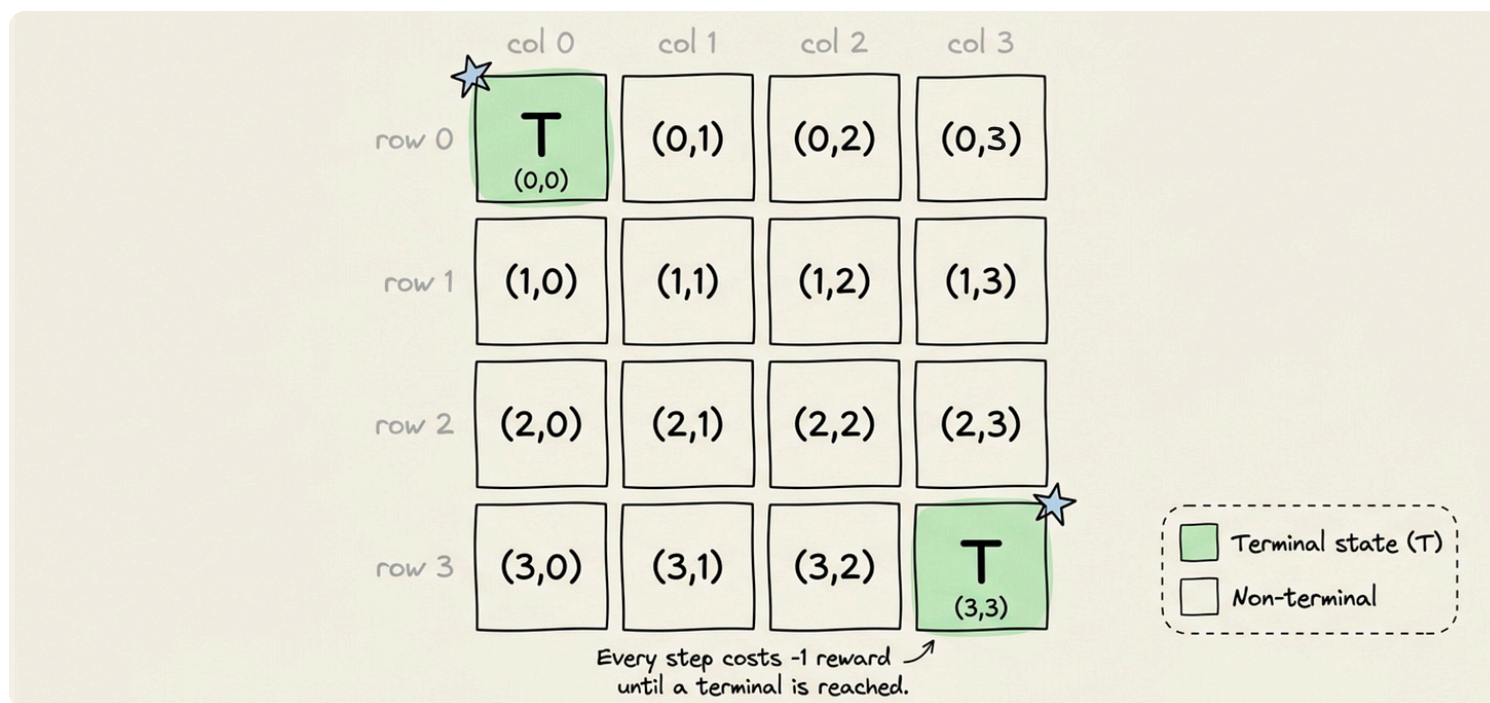
The environment

We use a 4×4 gridworld matching Sutton and Barto's Example 4.1 from the book. The grid has 16 cells. Two of them, the top-left at coordinate (0, 0) and the bottom-right at (3, 3), are terminal. The remaining 14 are non-terminal.

The agent has four actions: up, right, down, and left. Transitions are deterministic; if the chosen action would take the agent off the grid, it stays in

place (it bounces off the wall). Every transition gives reward -1 , until the episode ends when the agent reaches a terminal state.

This setup makes the value of every state intuitive. Under any policy, $v_{\pi}(s)$ is roughly the negative of the expected number of steps to reach a terminal state. The further from a terminal, the more negative the value.



Now let's look at the code:

The `GridWorld` class is the environment. A few things to note about how it is built:

- `__init__` sets up the grid size, the two terminal cells, and the action-to-direction mapping. `n_actions` exposes the number of actions for the policy code to use.
- `reset` rejection-samples a starting cell until one is non-terminal, then stores it as the current state. Sampling uniformly across non-terminals ensures every non-terminal state has positive probability of being an episode start, so in the limit of many episodes each one is visited as a start.
- `step` applies the action delta, clamps movement at the walls so the agent stays in place when it would otherwise leave the grid, returns

reward `-1.0` regardless of the transition, and marks `done = True` when the next state is terminal. The state is updated before returning.

The class deliberately has no notion of a policy, no value table, and no learning logic. It is purely the environment, kept clean so the MC code can interact with it through a standard `reset` / `step` loop without coupling.

The MC evaluator

Monte Carlo policy evaluation for a fixed policy on a gridworld:

This file has three pieces:

- `random_policy` is the equiprobable random policy, returning a uniformly-sampled action index for each call. No state-dependence: it samples the same way from every state. This is the policy whose value we are estimating.
- `run_episode` runs the standard interaction loop. From a randomly-chosen non-terminal start state, it picks an action via `random_policy`, calls `env.step`, records the `(state, reward)` pair, and continues until the episode terminates. The trajectory only contains non-terminal states with the reward emitted upon leaving them; the terminal state is never added because the loop breaks before that.

- `mc_policy_evaluation` is the actual estimator. It walks each episode backwards using the recursive definition of G_t . As it walks, it accumulates G for every visited state into `returns_sum`, and increments `returns_count`. After all episodes, it divides sum by count for each state and returns the resulting value table.

Note: This is every-visit MC, that is, if a state appears multiple times in an episode, every visit contributes to the average. There is also a first-visit variant that only counts the first occurrence per episode. Both converge to the same limit, with slightly different finite-sample variance, and the implementation differences are minor. We will not spend time on them here.

Both converge to v_π in the limit; they differ slightly in bias and variance for finite samples, but the practical difference is small here.

Running the experiment

Finally let's instantiate the gridworld and run the experiment:

The `main` function instantiates the gridworld and runs MC at three scales: 100, 1,000, and 10,000 episodes. After each run it prints the resulting value table rounded to two decimals.

Then, using the 10,000-episode estimates, it draws a heatmap with values overlaid on each cell and saves it to disk.



We use $\gamma = 1$ (undiscounted) because the gridworld is episodic and every episode terminates, so returns are bounded automatically. We seed the random number generator at 42 for reproducibility.

Results

Running `python main.py` produces the following output:

Value estimates after 100 episodes:

```
[[ 0.    -20.03 -26.94 -23.39]
 [-17.42 -20.56 -21.97 -18.75]
 [-26.55 -23.26 -17.63 -14.15]
 [-35.9  -27.82 -15.97  0.   ]]
```

Value estimates after 1000 episodes:

```
[[ 0.    -14.77 -20.14 -21.96]
 [-13.17 -17.71 -19.1  -19.34]
 [-20.34 -19.74 -16.62 -13.3  ]
 [-23.99 -20.47 -12.87  0.   ]]
```

Value estimates after 10000 episodes:

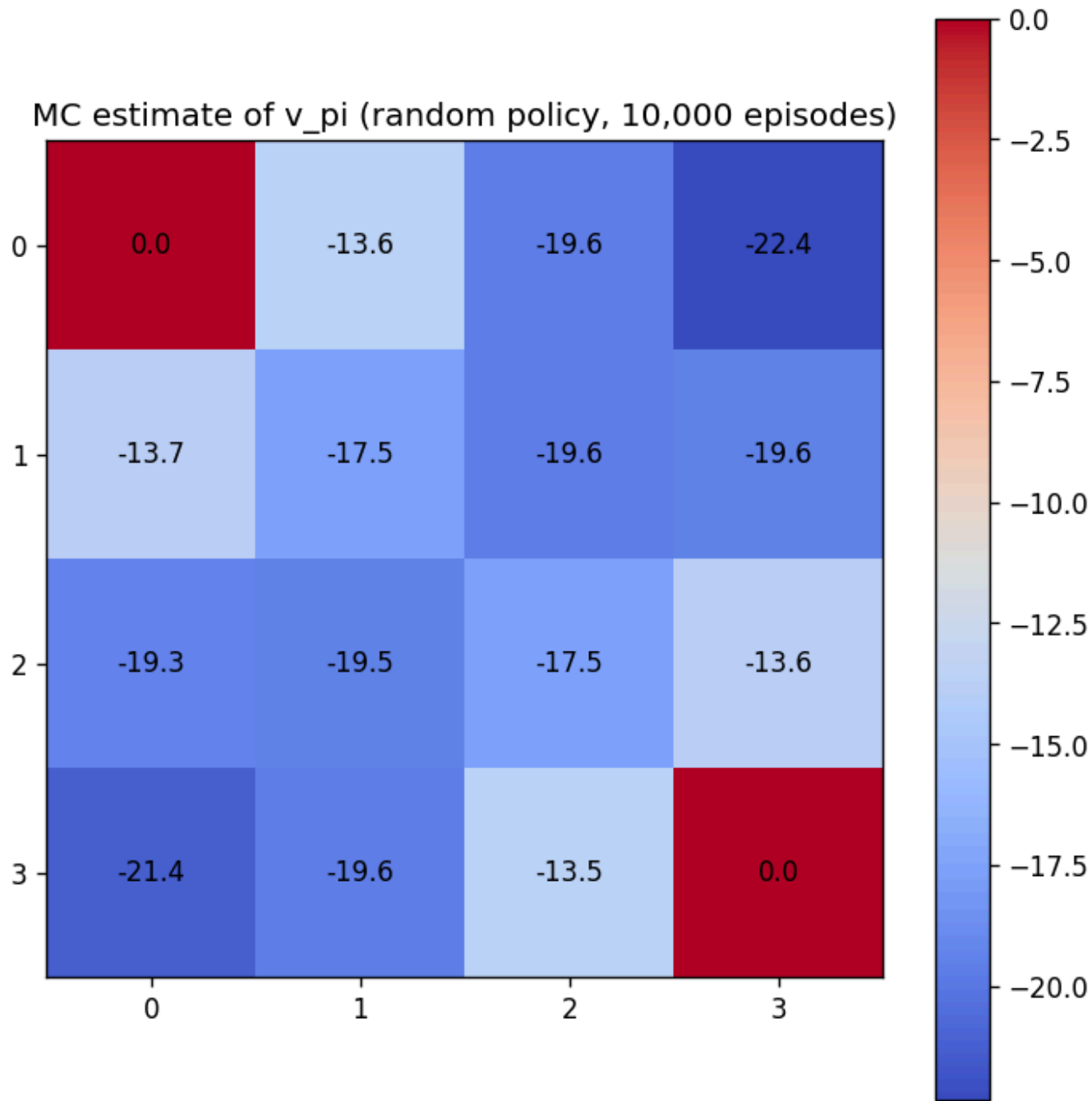
```
[[ 0.    -13.58 -19.64 -22.35]
 [-13.7  -17.49 -19.59 -19.55]
 [-19.3  -19.47 -17.5  -13.63]
 [-21.36 -19.61 -13.54  0.   ]]
```

Heatmap saved to value_heatmap.png

A few observations from the output stand out for this experiment:

- After only 100 episodes, the estimates are noisy. Some states have only been visited a handful of times, so their averages are far from the truth. The bottom-left corner shows -35.9 , well below the expected value.
- After 1,000 episodes, the structure has emerged. Cells adjacent to terminals are around -13 or -14 , the center cells lie between -17 and -20 , and the cells diagonally opposite from terminals are the most negative.
- After 10,000 episodes, the picture gets quite clear. Cells one step from a terminal at roughly -14 , central cells at roughly -18 to -20 , and the worst cells (the two diagonally opposite from terminals) at roughly -22 .

Additionally, a heatmap also gets saved:



Here we have visualized our experiment. Here also we can see that the two diagonally opposite cells from terminals are most negative.

This is concrete evidence that the definition $v_{\pi}(s)$ is computable. Given a policy and an environment we can simulate, and estimate the value of every state to any desired accuracy by running enough episodes. No closed-form solution required, no knowledge of P assumed.

That wraps up the hands-on section. We have moved from " v_{π} is an expectation" to a working implementation that produces concrete numbers.

With this we conclude the discussion for this chapter. In the upcoming chapters, we will continue to build on the core ideas of RL, and explore concepts and their implementations wherever applicable.

Conclusion

In this chapter, we explored the formal language that all of RL relies on.

We started with the Markov property, the assumption that the future depends on the past only through the present, and saw how it makes the problem tractable while also being a modeling choice that depends on how we define the state.

We then formalized the Markov decision process as a 5-tuple of states, actions, transition function, reward function, and discount factor.

We distinguished episodic and continuing tasks, and defined the return G_t . We discussed the reward hypothesis, its centrality to the framework, and its limits in real-world settings, where reward hacking is now a documented practical concern.

We then introduced policies, both deterministic and stochastic. We defined the state-value function and the action-value function, and noted the relationship between them.

Finally, we built a 4x4 gridworld from scratch and used Monte Carlo rollouts to estimate v_π .

In the next chapter, we will continue to build on the core ideas of RL and learn about the Bellman equation and dynamic programming methods.

The aim, as always, is to build a strong conceptual foundation and equip you with a flexible framework for reasoning about the core principles and the broader landscape in general.

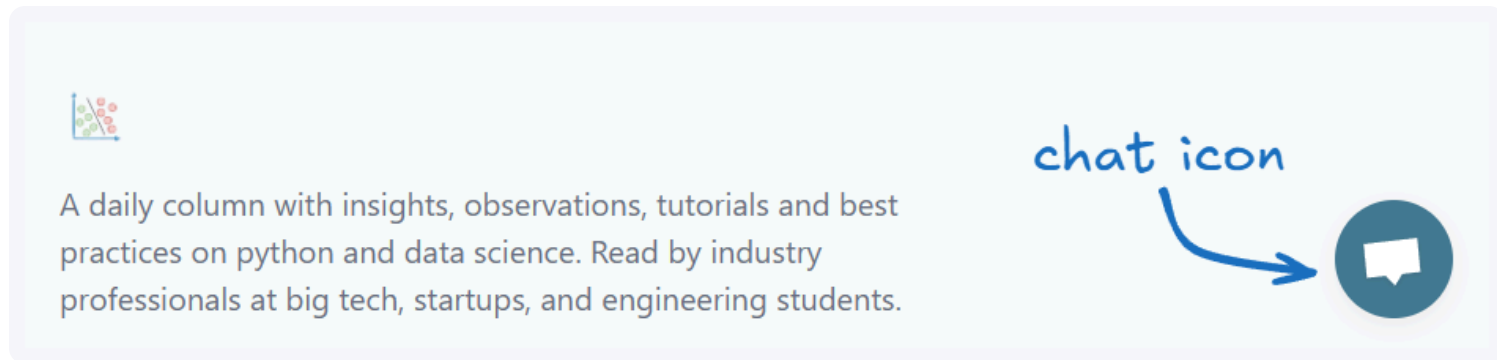
As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:



A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

chat icon

Published on May 3, 2026

 Comments

 Share

[Previous](#)[Next](#)[< Foundations of Reinforcement Learning](#)[Bellman Equations and Dynamic Programming >](#)

A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

Menu

[Contact](#)[FAQ](#)

Daily Dose of Data Science © 2026